Testing the Code

DevOps Best Practices for Continuous Delivery

Agenda

- Manual Testing Strategies
- Test Automation Frameworks
- Unit, Integration & Functional Testing
- CI/CD Integration with Jenkins

Why Testing Matters for DevOps

For Continuous Delivery

- Early bug detection
- Confidence in releases
- Faster feedback loops
- Reduced manual effort

Key Principle

"If we release code early and often, we must be confident of its quality through automated regression testing."

Manual Testing

Foundation for Automation

Requirements

- Manage test data consistently
- Rapid deployment capability
- Repeatable test results
- Clean database states

Best Practice

"Keep it simple, stupid" (KISS rule)

Test Automation: Pros & Cons

Benefits

- Higher software quality
- Increased release confidence
- Reduced manual testing burden
- Faster feedback cycles
- Better regression detection

Challenges

- Unit tests perceived as lower value
- Complex test fixtures required
- Tests need maintenance over time
- Difficult to write robust tests
- Risk of brittle test suites

Successful Test Automation Strategy

1. **Leverage team enthusiasm**
   Build on existing motivation for automation

2. **Set realistic goals**
   Don't aim for 100% coverage immediately

3. **Work incrementally**
   Add tests gradually as features develop

4. **Foster DevOps culture**
   Break down silos between dev and ops

Unit Testing

Testing Well-Defined Parts in Isolation

Key Concepts

- Test runner: Executes tests (e.g., Surefire)
- Test case: Fundamental test definition
- Test fixtures: Known state for tests
- Assertions: Verify expected conditions

JUnit Example

```
assertEquals(str1, str2);
assertTrue(val1 < val2);
assertFalse(val1 > val2);
```

xUnit Framework Family

SUnit (1998, Smalltalk) → JUnit (Java) → NUnit (C#/.NET)

Java	**JUnit**	Original and most widely adopted
C# / .NET	**NUnit**	Microsoft platform testing
Python	**PyTest / unittest**	Python standard library + extensions
JavaScript	**Jest / Mocha**	Modern JS/TS ecosystem

Mocking & Test Doubles

What is Mocking?

Simulating dependencies to enable unit testing without requiring actual resources like databases, APIs, or file systems.

- Also called: fakes, stubs, test doubles
- Framework: Mockito (Java/Python)
- Enables fast, isolated testing

Example

Middleware system responding with JSON from a database would 'mock' the database backend for unit tests.

Otherwise unit tests would require:

- Database to be online
- Exclusive access
- Complex setup/teardown

Test Coverage

Percentage of code executed by test cases

Tools

- Cobertura (Java)
- JCoverage
- Clover
- Coverage.py (Python)
- Istanbul (JavaScript)

Important Note

100% coverage is NOT always the goal. Consider cost/benefit tradeoffs. Simple getters/setters may not need tests.

Test methods with logic, not Java/Python implementation.

Automated Integration Testing

Testing with Real Components (Less Mocking)

Characteristics

- Uses real databases, services
- Tests full stack integration
- Requires predictable data
- Needs automated setup/teardown

Best Practices

- Use production data copies
- Automate backup/restore
- De-identify sensitive data
- Maintain repeatable results

Docker in Automated Testing

Isolation — Simulate entire clusters with containers

Networking — Virtual networks show container interactions

State Management — Easy restore to known state (like @After in unit tests)

CI Integration — Jenkins can start/stop containers automatically

```
Example: docker run -d --link selenium-hub:hub selenium/node-firefox
```

Performance Testing

Apache JMeter

Open source load testing tool

- Measures response times
- Generates synthetic load
- Protocols: HTTP, SOAP, JDBC, LDAP
- Maven plugin available
- Jenkins integration

Requirements

Production-like test environment needed for meaningful results

- Similar hardware specs
- Matching network topology
- Realistic data volumes
- Automated deployment to test env

Automated Acceptance Testing

Cucumber - Behavior-Driven Development (BDD)

Features

- Tests written in plain text
- Uses Gherkin language
- Non-programmers can write specs
- Brings roles together (DevOps!)
- Originally Ruby, now many ports

Example Feature

```
Scenario: Add numbers
  Given I enter 4
  When I press add
  And I enter 2
  Then result is 6
```

Automated GUI Testing

Selenium Web UI Testing Framework

How It Works

- Instruments controllers with IDs
- Finds elements via DOM
- Not blind mouse-click simulation
- Record interactions or write tests
- Works with multiple browsers

Selenium Grid

- Parallel test execution
- Multiple browser configurations
- Headless server support
- Docker containers available
- Scales test infrastructure

JavaScript Testing Frameworks

Karma *Test runner for unit tests*	Executes tests across multiple browsers
Jasmine *BDD testing framework*	Cucumber-like behavior testing for JS
Protractor *AngularJS testing*	Uses Selenium WebDriver under the hood
Jest *React testing framework*	Snapshot testing and component testing

Testing Backend Integration

SoapUI - SOAP & REST Testing

Why It Works Well

- Backend interfaces are stable
- Less maintenance than GUI tests
- Appeals to multiple roles
- GUI + command line support
- Maven integration available

DevOps Benefits

- Tests stored as XML (version control)
- Editable in text editor
- Global search/replace possible
- Open source licensing
- No license expiry issues

Test-Driven Development (TDD)

1. **Write the test first**
   Define interface and requirements
2. **Verify test fails**
   No implementation exists yet
3. **Write minimal code**
   Just enough to make test pass
4. **Verify test passes**
   Feature works, old tests still pass
5. **Refactor code**
   Clean up and optimize

Benefits: Good test suites for CI, better requirements understanding

Complete Test Automation Scenario

Example: User Database Web Application

Application Layers

- Web frontend
- JSON/REST service interface
- Application backend layer
- Database layer

Test Phases

- Unit testing (backend code)
- Functional testing (Selenium)
- REST interface testing (soapUI)
- All tests in sequence

When all tests succeed → Deploy to test environment for manual QA

CI/CD Pipeline Integration

Running Tests in Jenkins

1. Code commit triggers build
2. Maven runs unit tests automatically
3. Integration tests execute in Docker containers
4. Selenium tests run against deployed app
5. Performance tests measure response times
6. Test results published to Sonar/Jenkins dashboard
7. On success: Deploy to staging environment

Key Takeaways

Start Simple
Begin with manual testing, automate incrementally

Foster Collaboration
Break down silos between dev, QA, and ops teams

Multiple Test Types
Use unit, integration, functional, and performance tests

Leverage Tools
Docker, Jenkins, Selenium automate complex scenarios

Continuous Testing
Integrate all tests into CI/CD pipeline for early feedback

Conclusion

"Testing is completely necessary for a program to work reliably in the real world. Manual testing is too slow to achieve Continuous Delivery. We need test automation to succeed."